

02

Terminal で Python を 実行する

第 1 回の計算をファイルへ保存し、自分で動かす。

180 分 / 反転学習

PPT 発表なし

第 1 回から第 2 回へのつながり

計算内容は復習し、実行する場所とファイルの扱いを学ぶ。

01 前回：セルで変数・if・関数を試した

02 今回：.py を保存し、Terminal で実行する

03 共通：予想・変更・結果を自分で確かめる

180 分の使い方

Terminal を中心に、各自で実行・変更・確認する。

時間 [分]	活動
0-15	第 1 回の復習
15-35	Terminal を開く
35-60	場所とファイル
60-85	.py へ保存・実行
85-95	休憩
95-120	引数・保存・再実行
120-140	エラー・コンパイル
140-165	図と CSV を確認
165-180	記録・説明・提出

授業前

第 1 回の速度 × 時間、
変数・if・関数を見直す。

発表

PPT 発表なし

Colab の Terminal を開く

ランタイムに接続し、画面下部の Terminal を開く。

01 Terminal : pwd や python を入力する

02 Python セル : print や計算式を書く

03 >>> は Python 対話画面。exit() で戻る

場所とファイルを確認する命令

1 行ずつ入力し、表示された結果を Notebook へ記録する。

01 pwd : 今いる場所 ls : 中身の一覧

02 cd : 移動 mkdir -p : フォルダ作成

03 cat : 中身 cp : コピー mv : 名前変更

第 1 回の計算を .py へ保存する

速度 $1.2 \text{ m/s} \times$ 時間 5 s 。まず手計算してから確かめる。

01 Notebook の保存セルで .py を作る

02 Terminal の ls と cat でファイルを確認

03 編集後は保存し直してから実行する

Terminal から Python を実行する

Notebook で distance_demo.py を保存した後、Terminal へ入力する。

```
cd /content/b3_lesson02
pwd
ls
cat distance_demo.py
python distance_demo.py
python distance_demo.py 10
```

1 命令の意味

python : 実行するプログラム
.py : 読むファイル

2 入力を変える

10 : 時間の引数 [s]
予想と実際の距離を比べる

Terminal では ! や %%bash を付けない。プロンプトも入力しない。

エラーの原因を切り分ける

存在しないファイル名や数値でない入力を試し、直して再実行する。

01 No such file : pwd ・ ls ・ 保存を確認

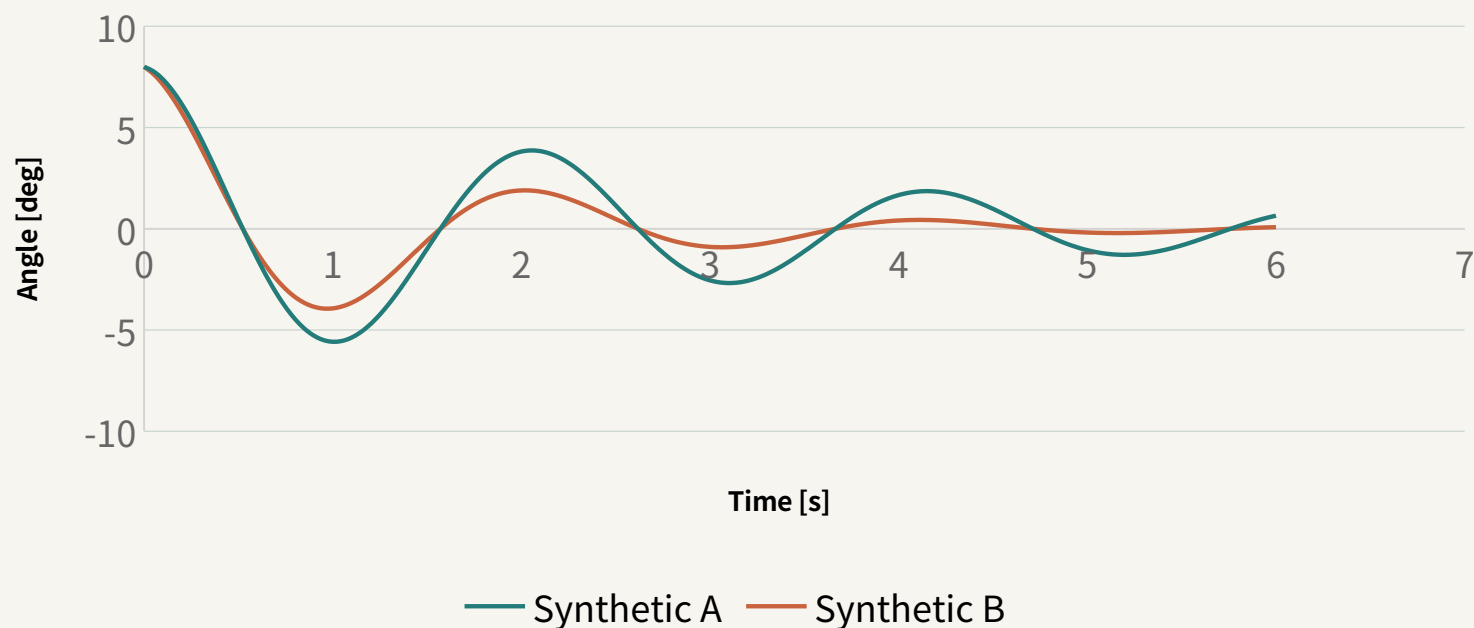
02 NameError : 変数や import の定義を確認

03 ValueError : 引数 five を 5 へ直して確認

実行例から、確認のしかたを学ぶ

教材の既定条件による例。自分の変更結果も確認する。

二つの合成時系列



実測・制御結果ではなく、
描画練習の架空データ。

凡例・単位・しきい値を
自分の図にも付ける。

実行とコンパイルの違い

通常は `python` で実行。 `py_compile` は構文を確認し、バイトコードを作る。

01 実行： `python distance_demo.py`

02 確認： `python -m py_compile distance_demo.py`

03 コンパイル成功後も、入力と出力を検証する

保存して、操作と結果を説明する

Terminal の履歴は Notebook へ自動保存されない。命令と出力を残す。

残すもの

- 1 T1-T5 の回答 ・ 命令 ・ 結果
- 2 distance_demo.py ・ result.txt
- 3 単位付きの図 ・ CSV

説明の順序

どこで、何を実行したか
何を変更したか
どんな結果だったか
エラーをどう直したか

PPT 発表なし

自分で実行・診断できるか

関数・配列・図・CSV は後半で確認する。Terminal の操作が中心。

ファイルを探し、Python を動かせるか。

保存・実行・コンパイルの違いを説明できるか。

次へ

第 3 回：この手順を使って MuJoCo を動かす。